

CITS3002: Lab 5 - Solution

Exercise 1:

In an IEEE 802.11 wireless network, the bit error probability is p . A station uses the RTS/CTS mechanism prior to transmitting a data frame. The transmission process consists of four frames in the following order: RTS, CTS, Data, and ACK. Let the sizes of these frames be L_{RTS} , L_{CTS} , L_{Data} , and L_{ACK} , respectively. Assume that no collisions occur and that transmission errors occur only due to bit errors.

a) Write the probability that each individual frame is received correctly.

Answer:

For an n -bit frame, the probability that it is received correctly is: $(1-p)^n$
So for each frame:

$$\begin{aligned}P(\text{RTS received correctly}) &= (1-p)^{L_{RTS}} \\P(\text{CTS received correctly}) &= (1-p)^{L_{CTS}} \\P(\text{Data received correctly}) &= (1-p)^{L_{DATA}} \\P(\text{ACK received correctly}) &= (1-p)^{L_{ACK}}\end{aligned}$$

b) Write the probability that the entire transmission exchange (RTS, CTS, Data, and ACK) is completed successfully without any bit error in one attempt.

Answer:

For the whole RTS/CTS/Data/ACK exchange:

$$\begin{aligned}P_{\text{success}} &= (1-p)^{L_{RTS}} \times (1-p)^{L_{CTS}} \times (1-p)^{L_{DATA}} \times (1-p)^{L_{ACK}} \\P_{\text{success}} &= (1-p)^{L_{RTS}+L_{CTS}+L_{DATA}+L_{ACK}}\end{aligned}$$

c) If $p = 10^{-4}$, $L_{RTS} = 160$ bits, $L_{CTS} = 112$ bits, $L_{DATA} = 12144$ bits, and $L_{ACK} = 112$ bits, compute the numerical probability of success.

Answer:

$$P_{\text{success}} = (1-10^{-4})^{160+112+12144+112} = 0.2856$$

So the full exchange succeeds with probability about 28.5%.

Exercise 2:

Write a Python program that implements Dijkstra's algorithm to find the shortest path from router A to every other node in the topology:

(A,B)=4, (A,C)=2, (B,C)=1, (B,D)=5, (C,D)=8, (C,E)=10, (D,E)=2, (D,F)=6, (E,F)=3

Answer:

The code should initialize all node distances to infinity (unknown) except the source A, then it repeatedly selects the non-permanent node with the smallest tentative distance, marks it as permanently labeled, and relaxes (updates) the distances of its neighbors if a shorter path is

found through the current node. It also keeps track of the previous node to reconstruct paths at the end. The permanent list represents nodes whose shortest distance is finalized, and the printed steps show how tentative and permanent labels evolve until all nodes are processed, producing the correct shortest paths from *A* to every other node. The complete implementation can be found in Lab5-Exercise2.py.

Exercise 3:

Assume node H is directly connected to nodes I, J, and K. The link costs from H to its neighbors are:

$$(H, I) = 2, \quad (H, J) = 3, \quad (H, K) = 6$$

At a certain time, node H receives the following distance vectors from its neighbors:

Vector of node I	
Dest	Cost
H	2
I	0
J	2
K	5
L	4
M	7

Vector of node J	
Dest	Cost
H	3
I	2
J	0
K	1
L	5
M	6

Vector of node K	
Dest	Cost
H	6
I	5
J	1
K	0
L	2
M	3

Using the distance vector routing, construct the routing table for node H.

Answer:

Destination I

- via I: $2 + 0 = 2$
- via J: $3 + 2 = 5$
- via K: $6 + 5 = 11$

Minimum: 2 via I

Destination J

- via I: $2 + 2 = 4$
- via J: $3 + 0 = 3$
- via K: $6 + 1 = 7$

Minimum: 3 via J

Destination K

- via I: $2 + 5 = 7$

- via J: $3 + 1 = 4$
- via K: $6 + 0 = 6$

Minimum: 4 via J

Destination L

- via I: $2 + 4 = 6$
- via J: $3 + 5 = 8$
- via K: $6 + 2 = 8$

Minimum: 6 via I

Destination M

- via I: $2 + 7 = 9$
- via J: $3 + 6 = 9$
- via K: $6 + 3 = 9$

Minimum: 9

H can choose I, J, or K. Usually one is chosen arbitrarily. We can select I.

Routing table of node H

Dest	Next hop	Cost
H	-	0
I	I	2
J	J	3
K	J	4
L	I	6
M	I	9